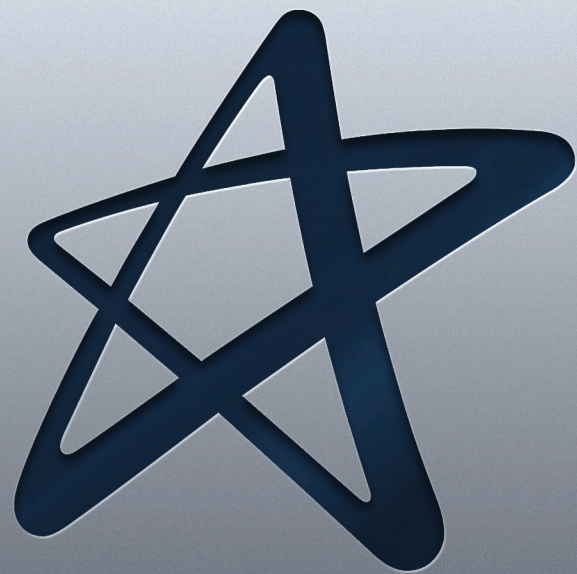


Computação Paralela e Distribuída



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Sistemas Operacionais, Componentes e Serviços

Responsável pelo Conteúdo:

Prof. Esp. Allan Piter Pressi

Revisão Textual:

Prof. Esp. Claudio Pereira do Nascimento

UNIDADE

Sistemas Operacionais, Componentes e Serviços



- Introdução ao Suporte a Sistemas Operacionais;
- Objetos Distribuídos e Componentes;
- *Web Services*.



OBJETIVO DE APRENDIZADO

- Apresentar aos alunos como é o funcionamento da interação e suporte para o sistema operacional;
- Compreender e conhecer como os sistemas operacionais oferecem suporte e os componentes necessários para serviços distribuídos.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução ao Suporte a Sistemas Operacionais

Este capítulo descreve como o *middleware* é suportado pelas instalações do sistema operacional em um sistema distribuído. O sistema operacional facilita o encapsulamento e proteção de recursos dentro de servidores e suporta os mecanismos necessários para acessar esses recursos, incluindo comunicação e agendamento.

Um ponto importante é o papel do *kernel* do sistema operacional. O capítulo visa dar uma compreensão das vantagens e desvantagens da divisão e funcionalidade entre os domínios de proteção - em particular, da funcionalidade de divisão entre código no nível do *kernel* e do usuário. Os *trade-offs* entre as instalações no nível do *kernel* e o nível do usuário das instalações são discutidas, incluindo a tensão entre eficiência e robustez. O capítulo examina o *design* e a implementação do processamento *multithreaded* e facilidades de comunicação, como também visa compreender e conhecer as principais arquiteturas de *kernel* e analisa o importante papel que a virtualização propõe aos sistemas operacionais.

Introdução

Já vimos as principais camadas de *software* em um sistema distribuído. Nós aprendemos que um aspecto importante dos sistemas distribuídos é o compartilhamento de recursos. Aplicativos clientes podem invocar operações em recursos que geralmente estão em outro local ou pelo menos em outro processo.

Aplicações e serviços usam a camada de *middleware* para suas interações. *Middleware* permite a comunicação remota entre objetos ou processos nos nós de um sistema distribuído.

Vamos conhecer os principais tipos de invocação remota encontrados no *middleware*, como Java RMI e CORBA, compreendendo as formas de comunicação. Vamos nos concentrar no suporte para comunicação, sem garantias em tempo real.

Abaixo da camada de *middleware* está a camada do sistema operacional (SO), que é o assunto deste capítulo. Vamos entender a relação entre os dois, e em particular, como os requisitos do *middleware* podem ser atendidos pelo sistema operacional.

Esses requisitos incluem acesso eficiente e robusto a recursos físicos e os flexibiliza para implementar uma variedade de políticas de gerenciamento de recursos. A tarefa de qualquer sistema operacional é fornecer abstrações orientadas aos recursos físicos subjacentes - os processadores, memória, redes, armazenamento e os meios de comunicação. Um sistema operacional como o *UNIX* (e suas variantes, como *Linux* e *Mac OS X*), ou *Windows*, fornecem ao programador, por exemplo, arquivos em vez de blocos de disco, e com *sockets* em vez de acesso bruto à rede. Ele assume os

recursos físicos em um único nó e gerência eles para apresentar essas abstrações de recursos através da interface de chamada do sistema.

Antes de começarmos sobre os detalhes do *middleware* do sistema operacional e seu papel de apoio, é útil obter alguma perspectiva histórica examinando dois conceitos do sistema que surgiram durante o desenvolvimento de sistemas distribuídos: sistemas operacionais de rede e sistemas operacionais distribuídos.

As definições variam, mas os conceitos por trás deles são algo como o seguinte: O *UNIX* e o *Windows* são exemplos de sistemas operacionais de rede. Eles têm um recurso de rede integrado a eles e, portanto, pode ser usado para acessar recursos remotos.

O acesso à rede é transparente para alguns tipos de recursos - não para todos. Por exemplo, através de um sistema de arquivos distribuídos, os usuários têm acesso transparente aos arquivos. Ou seja, muitos dos arquivos que os usuários acessam são armazenados remotamente em um servidor e isso é amplamente transparente para suas aplicações.

Mas a característica definidora é que os nós executados em uma rede operando o sistema retêm autonomia no gerenciamento de seus próprios recursos de processamento. Em outras palavras, existem várias imagens do sistema, uma por nó. Com um sistema operacional de rede, um usuário pode logar-se remotamente em outro computador, usando o recurso de conexão remota, por exemplo, e executar processos lá.

No entanto, enquanto o sistema operacional gerência os processos em execução em seu próprio nó, não gerência processos nos nós.

Por outro lado, pode-se imaginar um sistema operacional no qual os usuários nunca estão preocupados com o local de execução dos seus programas ou com a localização de quaisquer recursos.

O sistema operacional tem controle sobre todos os nós no sistema e localiza de maneira transparente novos processos em qualquer nó que seja adequado às suas políticas de agendamento.

Por exemplo, poderia se criar um novo processo no nó menos carregado do sistema para impedir que nós individuais se tornem excessivamente sobrecarregados.

Um sistema operacional que produz uma única imagem do sistema como essa para todos os recursos em um sistema distribuído é chamado de sistema operacional distribuído [Tanenbaum e Van Renesse 1985].

Middleware e sistemas operacionais de rede

Na verdade, não há sistemas operacionais em geral, somente sistemas operacionais de rede, como *UNIX*, *Mac OS* e *Windows*. É provável que isso continue sendo o caso, por dois motivos principais.

O primeiro é que os usuários investiram muito em seu *software* aplicativo, que geralmente atende às necessidades atuais de resolução de problemas; eles não vão adotar um novo sistema operacional que não irá executar as suas aplicações, independentemente das vantagens de eficiência que oferece.

A segunda razão contra a adoção de sistemas operacionais distribuídos é que usuários tendem a preferir ter um grau de autonomia para suas máquinas, mesmo em uma organização de *grid*.

A combinação de *middleware* e sistemas operacionais de rede fornece um equilíbrio aceitável entre a exigência de autonomia por um lado e acesso a recursos transparentes de rede, por outro.

O sistema operacional de rede permite que os usuários utilizem seus processadores de texto favoritos e outros aplicativos independentes. O *middleware* permite que eles aproveitem os serviços que se tornam disponíveis em seu sistema distribuído.

A camada do Sistema Operacional

Os usuários só ficarão satisfeitos se a combinação *middleware*-SO tiver um bom desempenho.

O *middleware* é executado em uma variedade de combinações de *hardware* e sistema operacional (plataformas) nos nós de um sistema distribuído. O sistema operacional em execução em um nó, em um *kernel* e nível de usuário associado de serviços como bibliotecas de comunicação fornece seu próprio sabor de abstrações de recursos locais de *hardware* para processamento, armazenamento e comunicação.

Middleware utiliza uma combinação desses recursos locais para implementar seus mecanismos de invocações entre objetos ou processos nos nós.

Kernels e Processos do servidor são os componentes que gerenciam recursos e apresentam aos clientes uma interface para os recursos. Como tal, convém conter ao menos os seguintes recursos:

- Encapsulamento;
- Proteção de Recursos;
- Processamento simultâneo;
- Comunicação;
- Agendamento.

As principais funcionalidades do sistema operacional com a qual devemos nos preocupar são: processamento e gerenciamento de *threads*, gerenciamento de memória e comunicação entre processos no mesmo computador.

Os principais componentes do sistema operacional e suas responsabilidades são:

1. **Gerente de processos:** criação e operações em processos.
2. **Gerenciador de *threads*:** criação de *threads*, sincronização e agendamento.

3. **Gestor de comunicação:** comunicação entre *threads* conectadas a diferentes processos no mesmo computador.
4. **Gerenciador de memória:** gerenciamento de memória física e virtual.
5. **Supervisor:** despacho de interrupções, *traps* de chamadas do sistema e outras exceções; ao controle de unidades de gerenciamento de memória e *caches* de *hardware*; processador e unidade de ponto flutuante, registrar manipulações.

Proteção

Dissemos acima que os recursos exigem proteção contra acessos ilegítimos. Contudo, Ameaças à integridade de um sistema não provêm apenas de código mal-intencionado.

Para entender o que é um “acesso ilegítimo” a um recurso, considere um arquivo e vamos supor, para fins de explicação, que os arquivos abertos tenham apenas duas operações, Ler e escrever. Proteger o arquivo consiste primeiro em garantir que cada uma das duas operações do arquivo possa ser realizada apenas por clientes com direito a executá-lo.

Kernels e proteção

O *kernel* é um programa que se distingue pelo fato de que permanece carregado a partir da inicialização do sistema e seu código é executado com privilégios de acesso para os recursos físicos em seu computador *host*. Em particular, pode controlar a unidade de gerenciamento de memória e definir os registros do processador para que nenhum outro código pode acessar os recursos físicos da máquina, exceto em formas aceitáveis.

A maioria dos processadores tem um registrador de modo de *hardware* cuja configuração determina se instruções privilegiadas podem ser executadas. Um *kernel* é o processo executado com o processador no modo privilegiado; o *kernel* organiza o que outros processos podem executar no modo usuário, ou seja, sem privilégios.

O *kernel* também configura espaços de endereço para proteger a si mesmo e outros processos e para fornecer os processos com seus recursos virtuais de memória. Um espaço de endereço é uma coleção de intervalos de locais de memória virtual, em cada uma das quais se aplica uma combinação específica de direitos de acesso à memória, como leitura ou leitura-gravação.

Quando um processo executa o código do aplicativo, ele é executado em um nível de usuário distinto em um espaço de endereçamento para esse aplicativo; quando o mesmo processo executa o código do *kernel*, executa-o no espaço de endereço do *kernel*.

O processo pode ser transferido de um nível de usuário para o *kernel* através de uma TRAP ou interrupção de memória utilizando uma chamada do sistema.

Processos e Encadeamentos

A noção tradicional de sistema operacional de que um processo executa uma única atividade era encontrado na década de 1980 para atender às exigências de sistemas distribuídos. O problema, como mostraremos, é que o processo tradicional faz compartilhamento entre atividades relacionadas, complicadas e caro.

A solução encontrada foi aprimorar a noção de um processo para que pudesse ser associado a várias atividades. Atualmente, um processo consiste em um ambiente de execução junto com um ou mais encadeamentos.

Uma *thread* no sistema operacional é uma abstração de uma atividade. O ambiente de execução é a unidade de gerenciamento de recursos.

Um ambiente de execução consiste principalmente de:

- um espaço de endereçamento;
- sincronização de *threads* e recursos de comunicação, como semáforos e interfaces de comunicação (por exemplo, *socket*);
- recursos de nível superior.

Os ambientes de execução normalmente são caros para criar e gerenciar, mas as várias *threads* podem compartilhá-las, isto é, elas podem compartilhar todos os recursos acessíveis dentro deles. Em outras palavras, um ambiente de execução representa o domínio de proteção no qual as *threads* são executadas.

Threads podem ser criadas e destruídas dinamicamente, conforme necessária. O objetivo central de ter vários encadeamentos de execução é maximizar o grau de execução simultânea entre operações, permitindo assim a sobreposição de computação com entrada e saída, e habilitar o processamento simultâneo em multiprocessadores.

Isso pode ser particularmente útil nos servidores, onde o processamento simultâneo de solicitações de clientes pode reduzir a tendência para os servidores se tornarem gargalos.

Espaços de Endereços

Um espaço de endereço é uma unidade de gerenciamento de uma memória virtual do processo e consiste em uma ou mais regiões, separadas por áreas inacessíveis de memória. Regiões não se sobrepõem. Cada região é especificada pelas seguintes propriedades:

- sua extensão (menor endereço virtual e tamanho);
- permissões de leitura/gravação/execução para os *threads* do processo;
- podem crescer.

A necessidade de compartilhar memória entre processos, ou entre processos e o *kernel*, é outro fator que leva ao uso de regiões no espaço de endereço. Uma memória compartilhada é aquela que é apoiada pela mesma memória física com uma ou mais regiões pertencentes a outros espaços de endereço.

Processos, portanto, podem ter acesso ao conteúdo de memória idêntica nas regiões que são compartilhadas, enquanto suas regiões não compartilhadas permanecem protegidas.

Os usos de regiões compartilhadas incluem os seguintes recursos:

- Bibliotecas;
- *Kernel*;
- Compartilhamento de dados e comunicação.

Criação de um novo processo

A criação de um novo processo tem sido tradicionalmente uma operação indivisível pelo sistema operacional. Por exemplo, a chamada do sistema no *UNIX* cria um processo com um ambiente de execução copiado do processo que o chamou.

A chamada do sistema do *UNIX* transforma o processo de chamada em um, executando o código de um programa nomeado.

Para um sistema distribuído, o *design* do mecanismo de criação de processos deve levar em conta a utilização de múltiplos computadores; consequentemente, o processo de suporte a infraestrutura é dividido em serviços de sistema separados.

A criação de um novo processo pode ser separada em dois aspectos independentes:

- a escolha de um *host* de destino;
- a criação de um ambiente de execução.

Como alternativa, o espaço de endereço pode ser definido em relação a um ambiente de execução. No caso da semântica do *UNIX*, por exemplo, o processo filho é criado e compartilha fisicamente a região de texto pai e tem regiões que são cópias da extensão do pai.

Threads

O próximo aspecto importante de um processo a ser considerado com mais detalhes são as *threads*.

Thread é um pequeno programa que trabalha como um subsistema, sendo uma forma de um processo se autodividir em duas ou mais tarefas. É o termo em inglês para Linha ou Encadeamento de Execução. Essas tarefas múltiplas podem ser executadas simultaneamente para rodar mais rápido do que um programa em um único bloco ou praticamente juntas, mas que são tão rápidas que parecem estar trabalhando em conjunto ao mesmo tempo.

As diversas *threads* que existem em um programa podem trocar dados e informações entre si e compartilhar os mesmos recursos do sistema, incluindo o mesmo espaço de memória. Assim, um usuário pode utilizar uma funcionalidade do sistema enquanto outras linhas de execução estão trabalhando e realizando outros cálculos e operações. É como se um usuário virtual estivesse trabalhando de forma oculta no mesmo computador que você ao mesmo tempo.

Devido à maneira rápida que a mudança de uma *thread* e outra acontece, aparentemente é como se elas estivessem sendo executadas paralelamente de maneira simultânea em *hardwares* equipados com apenas uma CPU. Esses sistemas são chamados de *monothread*. Já para os *hardwares* que possuem mais de uma CPU, as *threads* são realmente feitas concorrentialmente e recebem o nome de *multithread*.

As *threads* possuem vantagens e desvantagens ao dividir um programa em vários processos. Uma das vantagens é que isso facilita o desenvolvimento, visto que torna possível elaborar e criar o programa em módulos, experimentando-os isoladamente no lugar de escrever em um único bloco de código. Outro benefício das *threads* é que elas não deixam o processo parado, pois quando um deles está aguardando um determinado dispositivo de entrada ou saída, ou ainda outro recurso do sistema, outra *thread* pode estar trabalhando.

No entanto, uma das desvantagens é que com várias *threads* o trabalho fica mais complexo, justamente por causa da interação que ocorre entre elas.

Uma *thread* pode autorresponder-se sem que seja preciso duplicar um processo inteiro, economizando recursos como memória, processamento e aproveitando dispositivos de I/O, variáveis e outros meios. Também pode, por conta própria, abandonar a CPU por não ver a necessidade de continuar com o processamento proposto pela própria CPU ou pelo usuário. Isso é realizado por meio do método *thread-yield*. Uma *thread* também pode realizar outras funções além das mencionadas, como aguardar outra *thread* sincronizar, por exemplo, entre outras.

Os sistemas operacionais executam de maneiras diferentes os processos e *threads*. No caso do *Windows*, ele trabalha com maior facilidade para gerenciar programas com apenas um processo e diversas *threads* do que quando gerencia vários processos e poucas *threads*. Isso acontece porque no sistema da *Microsoft* a demora para criar um processo e alterná-los é muito grande. Enquanto isso, o *Linux* e os demais sistemas baseados no *Unix* podem criar novos processos de maneira muito mais rápidos. No entanto, ao serem alterados, os programas podem apresentar o mesmo desempenho tanto no *Linux* quanto no *Windows*.

Comunicação e Chamado Remoto

Na comunicação como parte da implementação do que temos chamado de invocação, uma construção, como uma invocação de método remoto, chamada de procedimento remota ou notificação de evento, cujo objetivo é realizar uma operação em um recurso em um espaço de endereço diferente.

Cobrimos questões e conceitos de *design* do sistema operacional, solicitando as seguintes perguntas sobre o sistema operacional:

- Quais os primitivos de comunicação que ele fornece?

- Quais protocolos ele suporta e quão aberta é a implementação da comunicação?
- Que medidas são tomadas para tornar a comunicação o mais eficiente possível?
- Qual suporte é fornecido para operação de alta latência e desconectada?

Primitivos de comunicação

Alguns *kernels* foram projetados para sistemas distribuídos e forneceu primitivas de comunicação adaptadas aos tipos de chamados. A economia na chamada do sistema sobrecarga são susceptíveis de ser ainda maior com a comunicação do grupo.

Na prática, o *middleware*, e não o *kernel*, fornece a maioria das instalações de comunicação encontradas em sistemas de hoje, incluindo RPC/RMI, eventos de notificação e comunicação em grupo.

Desenvolver *software* complexo como nível de usuário código é muito mais simples do que desenvolvê-lo para o *kernel*. Desenvolvedores normalmente implementam *middleware* sobre *sockets* que dão acesso a protocolos padrão da *Internet*, geralmente conectam soquetes usando TCP, mas às vezes soquetes UDP desconectados.

As principais razões para usar soquetes são portabilidade e interoperabilidade: o *middleware* é necessário para operar tantos sistemas operacionais amplamente utilizados quanto possível. Todos os sistemas operacionais comuns, como *UNIX* e a família *Windows*, fornecem APIs de soquete semelhantes, dando acesso a Protocolos TCP e UDP.

Protocolos

Um dos principais requisitos do sistema operacional é fornecer protocolos padrão que permitem o interfuncionamento entre o *middleware* em diferentes plataformas.

Dado o requisito diário de acesso à *Internet*, a compatibilidade no nível de TCP e UDP é exigido de todos os sistemas operacionais. E o sistema operacional ainda é necessário para permitir que o *middleware* aproveite os novos protocolos e recursos disponíveis no dispositivo.

Os protocolos são normalmente organizados em uma pilha de camadas. Muitos sistemas operacionais permitem que novas camadas sejam integradas estaticamente. Por contraste, o protocolo dinâmico é a composição técnica em que uma pilha de protocolos pode ser composta em tempo real para atender os requisitos de uma aplicação específica e utilizar qualquer uma das camadas físicas disponível.

Outro exemplo de composição dinâmica de protocolos é o uso de um pedido personalizado.

Performance de Chamados no Sistema

O desempenho de invocação ao sistema é um fator crítico no *design* de sistemas distribuídos. Clientes e servidores podem atender milhões de pedidos relacionados a chamadas operações.

As implementações de RPC e RMI têm sido objeto de estudo devido à aceitação generalizada desses mecanismos para clientes-servidores de uso geral em processamento.

Chamar um procedimento fazendo uma chamada de sistema, enviando uma mensagem por exemplo, pode gerar um custo de processamento que pode ser diferente em cada sistema e esses custos é associado ao tempo de processamento deste recurso. Os mecanismos de invocação podem ser síncronos ou assíncronas.

Os custos de invocação são importantes porque medem uma taxa fixa de sobrecarga, uma latência. Os custos de invocação aumentaram com os tamanhos dos itens e resultados, mas em muitos casos a latência é significativa em comparação com o restante do atraso.

Considere um RPC que procura uma quantidade de dados de um servidor. Tem um processo de concessão de um número inteiro, especificando o dado deve ser retornado. Toda a Chamada pode ter duas respostas possíveis, sucesso ou falha no atendimento a solicitação.

Operação assíncrona

Observamos como o sistema operacional pode ajudar a camada de *middleware* a fornecer mecanismos eficientes de invocação remota. Porém no contexto da *Internet* e os efeitos de uma latência e a baixa capacidade de transferência, neste contexto, um sistema operacional deve possuir uma arquitetura eficiente e prover benefícios que minimizem essas questões.

Podemos acrescentar a isso os fenômenos da rede como desconexão e reconexão, o que pode ser considerado como causador de comunicação de latência.

Uma técnica comum para derrotar altas latências é a operação assíncrona, que surge em dois modelos de programação: invocações simultâneas e assíncronas invocações. Estes modelos são em grande parte no domínio do *middleware*.

Fazendo invocações simultaneamente

No primeiro modelo, o *middleware* fornece apenas bloqueio de invocações, mas o aplicativo gera vários encadeamentos para realizar o bloqueio de invocações simultaneamente.

Um bom exemplo de tal aplicativo é um navegador da *web*. Uma página da *web* normalmente contém várias imagens. O navegador não precisa obter as imagens em uma sequência específica, por isso faz várias solicitações simultâneas por vez. Dessa forma, o tempo necessário para concluir todas as solicitações de imagens é tipicamente menor do que o atraso que resultaria ao fazer as solicitações em série.

Arquitetura de Sistemas Operacionais

Nesta seção, examinamos a arquitetura de um *kernel* adequado para um sistema distribuído.

Adotamos uma abordagem de primeiros princípios de começar com a exigência de abertura e examinando as principais arquiteturas de *kernel* que foram propostas.

Um sistema distribuído aberto deve possibilitar:

- executar somente o *software* do sistema em cada computador necessário para que ele execute seu papel particular na arquitetura do sistema;
- permitir que o *software* (e o computador) que implementa qualquer serviço específico seja independente de outras instalações;
- permitir que sejam fornecidas alternativas do mesmo serviço, quando necessário atender diferentes usuários ou aplicativos;
- introduzir novos serviços sem prejudicar a integridade dos já existentes.

Idealmente, o *kernel* forneceria apenas os mecanismos mais básicos sobre os quais as tarefas gerais de gerenciamento de recursos em um *host* são executadas. Módulos do servidor seriam dinamicamente carregados conforme necessário, para implementar o gerenciamento de recursos necessários de políticas para os aplicativos atualmente em execução.

Existem dois exemplos principais de *design* de *kernel*, as chamadas abordagens monolíticas e *microkernel*. Estes desenhos diferem principalmente na decisão sobre qual funcionalidade pertence ao *kernel* e o que deve ser deixado para o servidor de processos que podem ser carregados dinamicamente para serem executados sobre ele.

O *kernel* do sistema operacional UNIX tem sido chamado de monolítico. Este termo pretende sugerir que é massivo, ou seja, realiza todas as funções do sistema operacional, ocupa na ordem de *megabytes* de código e dados e é codificado de uma maneira não modular.

Por outro lado, no caso de um projeto de *microkernel*, o *kernel* fornece apenas abstrações básicas, principalmente espaços de endereços, *threads* e interprocessos locais de comunicação; todos os outros serviços do sistema são fornecidos por servidores que são dinamicamente carregados precisamente nos computadores do sistema distribuído que os requerem. Os clientes acessam esses serviços do sistema usando a invocação baseada em mensagem do *kernel* Mecanismos.

As principais vantagens de um sistema operacional baseado em *microkernel* são extensibilidade e sua capacidade de impor a modularidade por trás dos limites de proteção de memória.

Além disso, um *kernel* relativamente pequeno é mais provável que seja livre de *bugs* do que um que é maior e mais complexo.

A vantagem de um projeto monolítico é a eficiência relativa com a qual operações podem ser invocadas. As chamadas do sistema podem ser mais caras que os convencionais procedimentos, mas mesmo usando as técnicas que examinamos na seção anterior, uma invocação para um espaço de endereço separado no nível do usuário no mesmo nó é ainda mais dispendiosa.

O *kernel* coexiste em um único espaço de endereço com todo o sistema carregado dinamicamente de módulos e todas as aplicações. Quando carrega uma aplicação, o *kernel* coloca o código e dados do aplicativo em regiões escolhidas entre aquelas disponíveis no tempo de execução.

O advento dos processadores com endereçamento de 64 *bits* tornou o espaço de endereço os únicos sistemas operacionais particularmente atraentes, já que eles suportam espaços de endereços que pode acomodar muitos aplicativos.

Virtualização no nível do sistema operacional

A virtualização é um conceito importante em sistemas distribuídos. Nós já vimos uma aplicação da virtualização no contexto de redes, sob a forma de sobreposição de redes oferecendo suporte para classes específicas de aplicação. A virtualização também é aplicada no contexto de sistemas operacionais; de fato, é nesse contexto que a virtualização teve o maior impacto. Nesta seção, examinamos o que significa aplicar a virtualização no nível do sistema operacional (sistema virtualização) e, também, apresentam um estudo de caso do *Xen*, um exemplo líder de Virtualização.

Virtualização de Sistema

O objetivo da virtualização de sistemas é fornecer várias máquinas virtuais sobre a arquitetura da máquina física subjacente, com cada máquina executando uma instância separada do sistema operacional. O conceito deriva da observação de que as arquiteturas de computadores modernos têm o desempenho necessário para suporte potencialmente de um grande número de máquinas virtuais.

Várias instâncias do mesmo sistema operacional podem ser executadas nas máquinas virtuais ou uma gama de diferentes sistemas operativos pode ser suportada. O sistema de virtualização aloca o(s) processador(es) físico(s) e outros recursos de uma máquina física entre todas as máquinas virtuais que ele suporta.

Historicamente, os processos eram usados para compartilhar o processador e outros recursos entre várias tarefas em execução em nome de um ou vários usuários.

Surgiu mais recentemente e agora é comumente usado para essa finalidade. Oferece benefícios como segurança e separação clara de tarefas e na alocação e cobrança de cada usuário para seu uso de recursos com mais precisão do que pode ser alcançado com processos em execução em um sistema único.

Para entender completamente a motivação para a virtualização no nível do sistema operacional, é útil considerar diferentes casos de uso da tecnologia:

- Em máquinas servidoras, uma organização atribui cada serviço que oferece a uma máquina e, então, aloca de maneira ideal as máquinas virtuais para servidores físicos.
- A virtualização é muito relevante para o fornecimento de computação em nuvem, a computação em nuvem adota um modelo em que armazenamento, computação e objetos de nível superior construídos sobre eles são oferecidos como um serviço.
- Os desenvolvedores de soluções de virtualização também são motivados pela necessidade de aplicações distribuídas para criar e destruir máquinas virtuais prontamente e com pouca sobrecarga.
- Um caso bastante diferente surge ao fornecer acesso conveniente a vários ambientes do sistema operacional em um único computador *desktop*.

A virtualização do sistema é implementada por uma fina camada de *software* sobre a subjacente arquitetura da máquina física.

Isso tem a vantagem de que os sistemas operacionais existentes podem ser executados de forma transparente e não modificada no monitor da máquina virtual.



Em Síntese

Este capítulo descreveu como o sistema operacional suporta a camada de *middleware*, fornecendo invocações sobre recursos compartilhados. O sistema operacional fornece uma coleta de mecanismos sobre os quais podem ser adotadas várias políticas de gerenciamento de recursos, implementadas para atender aos requisitos locais e aproveitar as vantagens tecnológicas de melhorias. Permite que os servidores encapsulem e protejam recursos, enquanto permitem clientes para compartilhá-los simultaneamente. Também fornece os mecanismos necessários para os clientes invocar operações em recursos.

Um processo consiste em um ambiente de execução e encadeamentos: uma execução ambiente consiste em um espaço de endereço, interfaces de comunicação e outros recursos como semáforos; um segmento é uma abstração de atividade que executa dentro de um ambiente de execução. Os espaços de endereços precisam ser grandes e esparsos para suportar compartilhando e mapeando o acesso a objetos como arquivos. Os processos podem ter vários encadeamentos, que compartilham o ambiente de execução.

A invocação entre espaços de endereço dentro de um computador é um caso especial importante e descrevemos as técnicas de gerenciamento de *threads* e de passagem de parâmetros usadas com RPC leve.

A virtualização oferece uma alternativa atraente para esse estilo fornecendo emulação do *hardware* e, em seguida, permitindo várias máquinas virtuais (portanto, vários sistemas operacionais) para coexistir na mesma máquina.

Objetos Distribuídos e Componentes

Uma solução completa de *middleware* deve apresentar uma abstração de programação de nível superior bem e como abstrair sobre as complexidades subjacentes envolvidas em sistemas distribuídos.

CORBA é um projeto de *middleware* que permite que os programas aplicativos se comuniquem um com o outro, independentemente de suas linguagens de programação, seu *hardware* e plataformas de *software*, as redes que comunicam e os seus implementadores.

As aplicações são construídas a partir de objetos CORBA, que implementam interfaces definidas em Linguagem de definição de interface do CORBA, IDL. Como o *Java RMI*, o CORBA suporta invocação de métodos transparentes em objetos remotos. O componente de *middleware* que suporta RMI é chamado de intermediário de solicitação de objeto ou ORB.

O *middleware* baseado em componentes surgiu como uma evolução natural de objetos, fornecendo suporte para o gerenciamento de dependências entre componentes, ocultando detalhes de nível associados ao *middleware*, gerenciando as complexidades de estabelecer aplicativos distribuídos com propriedades não funcionais apropriadas, como segurança, e apoiar estratégias de implantação apropriadas.

Introdução

Os capítulos anteriores introduziram os fundamentos subjacentes fundamentais para sistemas distribuídos em termos de comunicação e suporte ao sistema operacional. Este capítulo volta-se para soluções completas de *middleware*, apresentando objetos distribuídos e componentes como dois dos estilos mais importantes de *middleware* em uso atualmente.

A tarefa do *middleware* é fornecer um nível de mais alto abstração de programação para o desenvolvimento de sistemas distribuídos e, através de camadas, abstrair sobre a heterogeneidade na infraestrutura subjacente para promover interoperabilidade e portabilidade.

A principal característica dos objetos distribuídos é que eles permitem que você adote um modelo de programação orientado a objeto para o desenvolvimento de sistemas distribuídos e, através disso, ocultar a complexidade subjacente de programação.

Objetos se comunicam principalmente usando invocação remota de método, mas também possivelmente usando um paradigma de comunicação alternativo. Isto é relativamente uma abordagem simples e tem vários benefícios importantes, incluindo os seguintes:

- O encapsulamento inerente às soluções baseadas em objetos é bem adequado para programação.

- A propriedade relacionada da abstração de dados fornece uma separação clara entre a especificação de um objeto e sua implementação, permitindo que os programadores apenas em termos de interfaces não se preocupem com detalhes de implementação como a linguagem de programação e o sistema operacional usados.
- Esta abordagem também se presta a soluções mais dinâmicas e extensíveis, por exemplo, permitindo a introdução de novos objetos ou a substituição de um objeto com outro objeto (compatível).

Soluções baseadas em componentes foram desenvolvidas para superar uma série de limitações que foram observadas para desenvolvedores de aplicativos trabalhando com *middleware* de objetos distribuídos.

As soluções baseadas em componentes podem ser melhor entendidas como uma evolução natural das abordagens baseadas na herança forte do trabalho anterior.

Objetos Distribuídos

O *middleware* baseado em objetos distribuídos é projetado para fornecer um modelo de programação com base em princípios orientados a objetos e, portanto, para trazer os benefícios da abordagem orientada à programação distribuída.

Em sistemas distribuídos, o *middleware* anterior era baseado no modelo cliente-servidor e havia um desejo por abstrações de programação mais sofisticadas, isso passa a surgir com o advento de linguagens com suporte a orientação à objetos.

Na engenharia de *software*, houve um progresso significativo no desenvolvimento de métodos de *design* orientados a objetos, levando ao surgimento da Linguagem de Modelagem (UML) como uma notação padrão industrial para especificar sistemas de *software* orientados a objetos.

Em outras palavras, através da adoção de uma abordagem orientada a objetos, sistemas distribuídos não são fornecidos apenas com abstrações de programação mais ricas, mas também são capazes de usar princípios, ferramentas e técnicas de *design* no desenvolvimento de *software* de sistemas distribuídos.

Uma coisa importante é o forte foco no encapsulamento e na abstração de dados, juntamente com os *links* poderosos para projetar metodologias. Devido às complexidades adicionadas envolvidas, o *middleware* de objeto distribuído deve fornecer funcionalidade adicional como:

- comunicação entre objetos;
- gerenciamento do ciclo de vida;
- ativação e desativação em implementações não distribuídas;
- persistência;
- serviços adicionais.

Objetos para Componentes

O *middleware* de objeto distribuído foi amplamente implementado em uma extensa gama de aplicações, incluindo as áreas como: finanças e comércio, cuidados de saúde, educação, transporte, logística e assim por diante.

No entanto, várias deficiências foram identificadas. Isso levou ao surgimento do que definiremos como abordagens baseadas em componentes, como evolução da computação de objetos distribuídos.

Componentes de *software* são como objetos distribuídos em que são encapsuladas unidades de composição, mas um dado componente especifica ambas as suas interfaces fornecidas ao mundo externo e suas dependências de outros componentes no ambiente distribuído.

As dependências também são representadas como interfaces. Mais um componente é especificado em termos de contrato, que inclui:

- um conjunto de interfaces fornecidas;
- um conjunto de interfaces necessárias.

Em uma determinada configuração de componente, cada interface necessária deve estar vinculada a uma interface fornecida de outro componente. Isso também é chamado de arquitetura de *software*, que consiste em componentes, interfaces e conexões entre interfaces.

Em particular, muitos objetos baseados em componentes abordagens oferecem dois estilos de interface:

- interfaces que suportam invocação remota de método, como no CORBA e *Java RMI*;
- interfaces que suportam eventos distribuídos.

A programação em sistemas baseados em componentes preocupa-se com o desenvolvimento de componentes e sua composição. O objetivo é apoiar um estilo de *software* de desenvolvimento paralelo ao desenvolvimento de *hardware* na utilização de componentes prontos para uso, compondo-os juntos para desenvolver serviços mais sofisticados e uma mudança no desenvolvimento de *software* para montagem de *software*. Uma gama de *middleware* baseado em componentes tecnologias emergentes, incluindo *Enterprise JavaBeans* e o Modelo de Componentes CORBA.



Em Síntese

Este capítulo examinou as questões do *middleware* baseadas em objetos e componentes distribuídos. Como agora será aparente, eles representam uma evolução em pensar sobre essas abstrações de programação. Objetos distribuídos são importante em termos de trazer os benefícios do encapsulamento e abstração de dados para sistemas distribuídos, bem como ferramentas e técnicas associadas do campo de *design* orientado a objetos. Objetos distribuídos, portanto, representam um avanço significativo de abordagens anteriores baseadas diretamente no modelo cliente-servidor. Ao aplicar a abordagem de objeto distribuído, no entanto, uma série de limitações surgiram. Em resumo, muitas vezes é também complexo na prática para usar soluções de *middleware* para aplicativos e serviços distribuídos.

As tecnologias de componentes superaram essas limitações, através de suas intrínsecas separações de interesses entre lógica de aplicação e gerenciamento de sistemas distribuídos.

A identificação explícita de dependências também ajuda em termos de suporte a terceiros na composição de sistemas distribuídos. As tecnologias de componentes são importantes para o desenvolvimento de aplicativos, mas como qualquer tecnologia, elas têm seus pontos fortes e fracos.

Web Services

Um *Web Service* fornece uma interface de serviço, permitindo que os clientes interajam com os servidores em uma maneira mais geral do que os navegadores *web*.

Clientes acessam as operações na interface de um serviço da *web* por meio de solicitações e respostas formatadas em XML e geralmente transmitidas sobre HTTP.

Mas, para serviços da *Web*, informações adicionais, incluindo codificação, comunicação dos protocolos em uso e os locais de serviço devem ser fornecidos. Os usuários precisam de um meio seguro para criar, armazenar e modificar documentos, trocando-os pela *Internet*. Os canais seguros do *Transport Layer Security* (TLS) não fornecem todos os requisitos necessários.

Os serviços da *Web* são cada vez mais importantes em sistemas distribuídos, eles suportam interoperabilidade em toda a *Internet* global, incluindo a área chave do *business-to-business* integração e também a cultura emergente de “*mashup*” que permite que desenvolvedores de *software* de terceiros possam se acoplar à um *software* ou serviços existentes.

Os serviços da *Web* também fornecem o *middleware* subjacente para a computação em nuvem e em grade.

Introdução

O crescimento da *Web* nas últimas duas décadas comprova a eficácia de usar protocolos simples através da *Internet* como base para um grande número de áreas de serviços e aplicações. Em particular, o protocolo HTTP permite que clientes de uso geral, chamados navegadores, visualizem páginas da *Web* e outros recursos.

No entanto, o uso de um navegador de uso geral como cliente, mesmo com os aprimoramentos fornecidos por *applets* específicos de aplicativo baixados, restringe escopo potencial das aplicações. No modelo cliente-servidor original, cliente e servidor eram funcionalmente especializados. Os serviços da *Web* retornam a esse modelo, no qual um cliente específico da aplicação interage com um serviço por meio de uma funcionalidade fácil e interface através da *Internet*.

Assim, os serviços da *Web* fornecem uma infraestrutura para manter um conteúdo mais rico e uma forma mais estruturada de interoperabilidade entre clientes e servidores. Eles fornecem uma base pelo qual um programa cliente em uma organização pode interagir com um servidor em outra organização sem supervisão humana. Em particular, os serviços da *Web* permitem aplicações a serem desenvolvidas fornecendo serviços que integram vários outros serviços.

Devido à generalidade de suas interações, os serviços da *Web* não podem ser acessados diretamente em navegadores. Representação de dados externos e empacotamento de mensagens trocadas entre clientes e serviços da *web* são feitos em XML, que é uma representação textual que, embora mais volumosa, foi adotado para sua legibilidade e consequente facilidade de depuração.

O protocolo SOAP especifica as regras para usar o XML no pacote mensagens, por exemplo, para suportar um protocolo de solicitação-resposta. Os serviços e aplicativos da *Web* podem ser construídos em cima de outros serviços da *Web*. Alguns serviços da *Web* específicos fornecem a funcionalidade geral necessária para operação de um grande número de outros serviços da *Web*. Eles incluem serviços de diretório, segurança, criptografia e microserviços.

Um serviço da *Web* geralmente fornece uma descrição que inclui uma definição de interface e outras informações, como a URL do servidor. Isso é usado como base para um entendimento comum entre cliente e servidor quanto ao serviço oferecido.

Outra necessidade comum no *middleware* é que um serviço de nomenclatura ou diretório permita clientes obter informações sobre serviços.

A segurança no uso do XML é apresentada como documentos e podem ser assinados ou criptografados. Um documento que tenha sido assinado ou contenha elementos criptografados pode então ser transmitidos ou armazenados.

Os serviços da *Web* fornecem acesso a recursos para clientes remotos, mas eles não fornecer meios para coordenar suas operações entre si.

Webservices

Uma interface de serviço da *web* geralmente consiste em uma coleção de operações que podem ser usadas por um cliente pela *Internet*. As operações em um serviço da *web* podem ser fornecidas por uma variedade de recursos diferentes, por exemplo, programas, objetos ou bancos de dados. Uma teia de serviço pode ser gerenciada por um servidor da *Web* junto com páginas da *Web*; ou pode ser totalmente um serviço separado.

A principal característica da maioria dos serviços da *Web* é que eles podem processar mensagens formatadas. Uma alternativa é a abordagem que cada serviço da *Web* usa seu próprio serviço de descrição para lidar com as características específicas do serviço das mensagens que recebe.

Muitos servidores da *Web* comerciais conhecidos, incluindo *Amazon*, *Yahoo*, *Google*, etc., oferecem interfaces de serviço da *Web* que permitem que os clientes manipulem via *web* seus recursos.

Outro exemplo dos benefícios da combinação de vários serviços: considere o fato de que muitas pessoas reservem voos, hotéis e aluguéis de carros para viagens de forma online usando uma variedade de *sites* diferentes. Se cada um desses *sites* fornecesse uma interface de serviço *web* padrão, então um “serviço de agente de viagens” poderia usar suas operações e fornecer ao viajante uma combinação desses serviços.

Mais geralmente, os serviços da *Web* são projetados para suportar computação distribuída na *Internet*, em que muitas linguagens de programação e paradigmas diferentes coexistem.

SOAP

O SOAP foi projetado para permitir a interação cliente-servidor de forma assíncrona ao longo do *Internet*. Ele define um esquema para usar XML para representar o conteúdo da solicitação e mensagens de resposta, bem como um esquema para a comunicação de documentos. Originalmente, o SOAP era baseado apenas em HTTP, mas a versão atual é projetada para usar uma variedade de protocolos de transporte, incluindo SMTP, TCP ou UDP.

A especificação SOAP indica:

- como o XML deve ser usado para representar o conteúdo de mensagens individuais;
- como um par de mensagens únicas pode ser combinado para produzir um padrão de resposta de solicitação;
- as regras de como os destinatários das mensagens devem processar os elementos XML que eles contêm;

- como HTTP e SMTP devem ser usados para comunicar mensagens SOAP. Isto é, espera-se que futuras versões da especificação definam como usar outros protocolos de transporte, por exemplo, TCP.

Esta seção descreve como o SOAP usa XML para representar mensagens e HTTP para comunicá-los. No entanto, o programador normalmente não precisa se preocupar com esses detalhes, já que as APIs SOAP foram implementadas em muitos programas idiomas, incluindo *Java*, *JavaScript*, *Perl*, *Python*, *.NET*, *C*, *C ++*, *C #* e *Visual Basic*.

Para suportar a comunicação cliente-servidor, o SOAP especifica como usar o Método *POST* para a mensagem de solicitação e sua resposta para a mensagem de resposta. O uso combinado de XML e HTTP fornece um protocolo padrão para cliente-servidor comunicação pela *Internet*.

Descrições de serviço para serviços da web

As definições de interface são necessárias para permitir que os clientes se comuniquem com os serviços. Para *web* serviços, as definições de interface são fornecidas como parte de uma descrição de serviço mais geral, que especifica duas outras características adicionais de como as mensagens devem ser comunicadas e o URI do serviço. Para atender para uso em um ambiente de vários idiomas, as descrições de serviço são gravadas em XML.

Uma descrição de serviço forma a base de um acordo entre um cliente e um servidor quanto ao serviço oferecido. Reúne todos os fatos relativos ao serviço que são relevantes para seus clientes.

A capacidade de especificar o URI de um serviço como parte da descrição do serviço evita a necessidade de um diretório separado ou serviço de nomes usados pela maioria dos outros *middleware*. Tem a implicação de que o URI não pode ser alterado uma vez que o serviço a esta descrição foi disponibilizado a potenciais clientes, mas o esquema URN atende para uma mudança de localização, permitindo uma indireção no nível de referência.

No contexto de serviços da *Web*, a WSDL (*Web Services Description Language*) é comumente usada para descrições de serviço. Ele define um esquema XML para representar os componentes de uma descrição de serviço, que incluem, por exemplo, as definições de nomes de elementos, tipos, mensagens, interface, ligações e serviços. O WSDL separa a parte abstrata de uma descrição de serviço da peça de concreto.

Um serviço de diretório para uso com serviços da web

Há muitas maneiras pelas quais os clientes podem obter descrições de serviço. Por exemplo, qualquer pessoa que forneça um serviço *Web* de nível superior, como o serviço de agente de viagens discutido anteriormente, quase certamente criaria uma página da *web* anunciando o serviço para clientes potenciais se depararem com a página da *web* ao procurar por serviços desse tipo.

No entanto, qualquer organização que planeje basear seus aplicativos em serviços da *Web* pode achar mais conveniente usar um serviço de diretório para disponibilizar esses serviços aos clientes. Este é o propósito da Descrição, Descoberta e Integração Universal serviço (UDDI), que fornece um serviço de nomes e um serviço de diretório.

Isto é, descrições de serviço WSDL podem ser por nome ou por atributo. Eles podem também ser acessados diretamente por meio de suas URLs, o que é conveniente para desenvolvedores que projetam programas clientes que usam o serviço.

Os clientes podem usar a abordagem de páginas amarelas para procurar uma categoria específica de serviço, como agente de viagens ou vendedor de livros, ou eles podem usar a abordagem de procure um serviço com referência à organização que o fornece.

Segurança XML

A segurança XML consiste em um conjunto de projetos relacionados ao W3C para assinatura, gerenciamento de chaves e criptografia. Destina-se ao uso em trabalho cooperativo pela *Internet*, envolvendo documentos cujo conteúdo pode precisar ser autenticado ou criptografado. Tipicamente, os documentos são criados, trocados, armazenados e depois trocados novamente, possivelmente, após ser modificado por uma série de diferentes usuários.

Como exemplo de um contexto no qual a segurança XML seria útil, considere um documento contendo os registros médicos de um paciente. Diferentes partes deste documento são usados na cirurgia do médico local e nas várias clínicas e hospitais especiais visitados pelo paciente. Será atualizado por médicos, enfermeiros e consultores que fazem anotações sobre a condição e tratamento do paciente, pelos administradores que marcam consultas e farmacêuticos que fornecem medicamentos. Diferentes partes do documento serão visualizadas pelos diferentes papéis mencionados acima e, possivelmente, pelo paciente também. É essencial que certas partes do documento, por exemplo, com recomendações quanto ao tratamento, possam ser atribuído à pessoa que o fez e pode ser garantido que não seja alterado.

A segurança XML define os elementos que podem ser usados para especificar a segurança através da URI do Algoritmo que foi utilizado para assinatura ou criptografia. O padrão para criptografia em XML é definido em um W3C recomendação que especifica tanto a maneira de representar dados criptografados em XML quanto processo para criptografá-lo e descriptografá-lo.

Coordenação de *Webservices*

A infraestrutura SOAP suporta interações de solicitação-respostas únicas entre clientes e serviços da *web*. No entanto, muitos aplicativos úteis envolvem várias solicitações que precisam para ser feito em uma ordem particular. Por exemplo, ao reservar um voo, o preço e informações de disponibilidade são coletadas antes que as

reservas sejam feitas. Quando um usuário interage com páginas da *web* por meio de um navegador, por exemplo, para reservar um voo ou fazer um lance em um leilão, a interface fornecida pelo navegador (que é baseado em informações fornecidas pelo servidor) controla a sequência em que as operações são executadas.

No entanto, se é um serviço da *web* que está fazendo reservas, o serviço de agente de viagens, o serviço da *Web* precisa trabalhar a partir de uma descrição da maneira apropriada de proceder ao interagir com outros serviços em que são utilizados, por exemplo, para contratar e reservar hotéis, bem como reservas de voos.

Este exemplo ilustra a necessidade de serviços da *Web* como clientes a serem fornecidos com uma descrição de um protocolo específico a seguir e ao interagir com outros serviços da *Web*.

Mas há também a questão de manter a consistência nos dados do servidor quando recebe e responde a solicitações de vários clientes.

O W3C usa o termo coreografia para se referir a uma linguagem baseada em WSDL para definição de uma coordenação. Por exemplo, o idioma pode especificar restrições no pedido e as condições em que as mensagens são trocadas pelos participantes.

Aplicações de serviços web

Os serviços da *Web* são hoje um dos paradigmas dominantes da programação de sistemas. Nesta seção, discutiremos várias das principais áreas em que os serviços da *Web* têm sido empregados extensivamente: no suporte à arquitetura orientada a serviços, o *Grid* e, ultimamente, computação em nuvem.

Arquitetura Orientada a Serviços

A arquitetura orientada a serviços (SOA) é um conjunto de princípios de *design* que distribui sistemas desenvolvidos usando conjuntos de serviços fracamente acoplados, que podem ser dinamicamente descoberto e, em seguida, comunicar uns com os outros, podendo ser coordenados através de coreografia para fornecer serviços aprimorados.

Arquitetura orientada a serviços é um resumo conceito que pode ser implementado usando uma variedade de tecnologias, incluindo o objeto distribuído ou abordagens baseadas em componentes.

Os principais meios de realizar arquitetura orientada a serviços, no entanto, é através do uso de *web* serviços, em grande parte devido ao fraco acoplamento inerente a esta abordagem (como já discutido nesta unidade).

Esse estilo de arquitetura pode ser usado em uma empresa ou organização para oferecer arquitetura de *software* flexível e para alcançar a interoperabilidade entre os vários Serviços.

Isso torna possível transcender os níveis de heterogeneidade inerentes à *Internet* e também para lidar com o problema de diferentes organizações que adotam diferentes produtos de *middleware* utilizando serviços da *Web*, assim incentivam a interoperabilidade global.

A arquitetura orientada a serviços também permite e incentiva uma abordagem de *mashup* desenvolvimento de *software*. Um *mashup* é um novo serviço criado por um desenvolvedor de terceiros combinando dois ou mais serviços disponíveis no ambiente distribuído.

Grid

O nome “*Grid*” é usado para se referir ao *middleware* projetado para permitir o compartilhamento de recursos como arquivos, computadores, *software*, dados e sensores em grande escala.

Os recursos são compartilhados tipicamente por grupos de usuários em diferentes organizações que estão colaborando na solução de problemas que exigem grandes números de computadores para resolvê-los, seja pelo compartilhamento de dados ou pelo compartilhamento do poder de computação.

Estes recursos são necessariamente suportados por *hardware* de computador heterogêneo, sistemas, linguagens de programação e aplicativos. A gestão é necessária para coordenar o uso de recursos para garantir que os clientes obtenham o que precisam e que os serviços possam pagar para fornecê-lo. Em alguns casos, são necessárias técnicas de segurança sofisticadas para garantir que o uso correto seja feito de recursos nesse tipo de ambiente.

Computação em nuvem

A computação em nuvem é como um conjunto de aplicativos baseados na *Internet*, serviços de armazenamento e computação suficientes para suportar as necessidades da maioria dos usuários, dispensando, em grande parte ou totalmente, o armazenamento local de dados e *software* de aplicação.

A computação em nuvem promove uma visão de que todos os recursos podem ser utilizados como um serviço e a partir de infraestrutura virtual através de *software*, muitas vezes pagos por uso, em vez de ser comprado.

O conceito está, portanto, intrinsecamente ligado a um novo modelo de negócios para Computação em que os fornecedores da nuvem oferecem uma gama de serviços computacionais, de dados e outros aos clientes, conforme necessário para o uso diário, por exemplo, oferecendo armazenamento suficiente com capacidade através da *Internet* para atuar como um serviço de arquivamento ou *backup*.

O desenvolvimento do *Grid* precedeu o surgimento da computação em nuvem e foi um fator significativo em sua emergência. Eles compartilham o mesmo objetivo de fornecer recursos (serviços) lá fora, na *Internet* maior. Considerando que o *Grid* tende a focar em *high-end* aplicações com muitos dados ou computacionalmente

dispendiosas, a computação na nuvem é mais geral, oferecendo uma variedade de serviços para usuários individuais de computadores Comerciais.

O modelo de negócios associado à computação em nuvem também é um diferencial como característica. Por isso, é justo dizer que o *Grid* é um dos primeiros exemplos de nuvem computação, mas a computação em nuvem se desenvolveu significativamente desde então.

Com a visão de tudo como um serviço, os serviços da *Web* oferecem um caminho de implementação da computação em nuvem e, de fato, muitos fornecedores seguem esse caminho.



Em Síntese

Neste capítulo, mostramos que os serviços da *Web* surgiram da necessidade de fornecer uma infraestrutura para apoiar o interfuncionamento entre diferentes organizações. Esta infraestrutura geralmente usa o protocolo HTTP amplamente utilizado para transportar mensagens entre clientes servidores pela *Internet* e baseia-se no uso de URLs para se referir a recursos.

XML, um formato textual, é usado para representação de dados e empacotamento. Duas influências separadas levaram ao surgimento de serviços da *web*. Um deles foi a adição de interfaces de serviço para servidores *web* com vista a permitir os recursos em um *site* para ser acessado por programas clientes que não sejam navegadores e que usa um formulário mais rico de interação. O outro era o desejo de fornecer algo como o RPC pela *Internet*, com base nos protocolos existentes. Os serviços da *Web* resultantes fornecem interfaces com conjuntos de operações que podem ser chamadas remotamente. Como qualquer outra forma de serviço, um serviço da *web* pode ser o cliente de outro serviço da *Web*, permitindo que um serviço da *Web* integre ou combine um conjunto de outros serviços da *web*.

O SOAP é o protocolo de comunicação geralmente usado pelos serviços da *Web* e seus clientes. Pode ser usado para transmitir mensagens de solicitação e suas respostas entre o cliente e servidor seja pela troca assíncrona de documentos ou por uma forma de solicitação que responda o protocolo com base em um par de trocas de mensagens assíncronas. Em ambos os casos, mensagem de pedido ou resposta é incluída em um documento formatado em XML chamado envelope. O envelope SOAP é geralmente transmitido através do HTTP síncrono protocolo, embora outros transportes possam ser usados.

Processadores XML e SOAP estão disponíveis para toda a programação amplamente utilizada com idiomas e sistemas operacionais. Isso permite que os serviços da *Web* e seus clientes sejam implantado em quase qualquer lugar. Esta forma de interação é permitida pelo fato de que *web* serviços não estão vinculados a nenhuma linguagem de programação específica e não suportam modelo de objeto distribuído.

Nos serviços de *middleware* convencionais, as definições de interface fornecem aos clientes detalhes dos serviços. No entanto, no caso de serviços da *Web*, as descrições de serviço são usadas.

Uma descrição do serviço especifica o protocolo de comunicação a ser usado (por exemplo, SOAP) e o URI do serviço, além de descrever sua interface. A interface pode ser descrita como um conjunto de operações ou como um conjunto de mensagens a serem trocadas entre cliente e servidor.

A segurança XML foi projetada para fornecer a proteção necessária para o conteúdo de um documento trocado por membros de um grupo de pessoas, que têm tarefas diferentes para executar nesse documento. Diferentes partes do documento estarão disponíveis para diferentes pessoas, algumas com a capacidade de adicionar ou alterar o conteúdo e outros apenas para lê-lo. Para permitir total flexibilidade em seu uso futuro, as propriedades de segurança são definidas com documentos em si. Isto é conseguido por meio de XML, que é um formato autoexplicativo.

Elementos XML são usados para especificar partes do documento que são criptografadas ou assinadas também como detalhes dos algoritmos usados e informações para ajudar a encontrar chaves.

Os serviços da *Web* foram usados para diversas finalidades em sistemas distribuídos. Por exemplo, os serviços da *Web* fornecem uma implementação natural do conceito de serviço, arquitetura orientada, na qual o seu acoplamento solto permite a interoperabilidade em aplicações de escala - incluindo aplicações *business-to-business* (B2B). Seu inerente acoplamento flexível também suporta o surgimento de uma abordagem de *mashup* para o serviço da *Web* construção. Os serviços da *Web* também sustentam o *Grid*, apoiando colaborações entre cientistas ou engenheiros em organizações de diferentes partes do mundo. Seu trabalho é muitas vezes com base no uso de dados brutos coletados por instrumentos em diferentes locais e, em seguida, processado localmente.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:



Sites

Wikipedia

Sistemas Distribuídos.

<https://goo.gl/b2g1EY>

Unesp

Sistemas Distribuídos – Conceitos.

<https://goo.gl/UTrmRc>

Wikilivros

Sistemas Distribuídos – Serviços.

<https://goo.gl/4Ci1RH>



Leitura

Componentes Distribuídos

<https://goo.gl/c6S3WP>

Referências

GAGLIARDI, Gary. **Cliente/servidor**. São Paulo: Makron Books do Brasil, 1996.

STALLINGS, William. **Arquitetura e organização de computadores**: projeto para o desempenho. 8. ed. São Paulo: Pearson Prentice Hall, 2009. ISBN 9788576055648.

TANENBAUM, Andrew S.; Steen, Maarten van. **Sistemas Distribuídos**: princípios e paradigmas. 2.ed. São Paulo: Pearson Prentice Hall, 2008. 416 ISBN 9788576051428.



Cruzeiro do Sul
Educatonal